

CCOS Gap Implementation Brief for Claude Code

Oct 8, 2026 · @yvesdlf

1. Purpose and how to use this brief

This brief tells Claude Code to close the feature gaps found by comparing CCOS against leading open-source projects in recipes (Tandoor, Mealie), inventory (InvenTree), procurement and finance (ERPNext, Bigcapital, MatchFlow, TaxHacker) and food safety (FoodSafe Tracker, haccpFlow). Those projects were studied for features only. No code is to be copied from them.

Work in five phases, in order. Each phase builds on the tables of the one before. Finish a phase, pass its checks in section 8, commit, update `docs/PROGRESS.md`, then start the next.

Before writing any code, Claude Code must:

1. Read `CLAUDE.md`, the CCOS Design Bible, `docs/PROGRESS.md`, the ADRs and the eight specification documents in `docs/`.
2. Read all existing Supabase migrations (products, recipes, organization hierarchy, authorization, self-approval-ban trigger) and the TypeScript Cost Engine in `packages/shared`.
3. Reconcile this brief with what already exists. Table and column names here are proposals: reuse existing tables, naming conventions and enums wherever they already cover the need, and extend rather than duplicate.
4. Write a short plan for the current phase into `docs/PROGRESS.md` (new migrations, new screens, engine changes), then implement it directly without waiting for approval.
5. Record any decision that changes the architecture as a new ADR.

Where this brief conflicts with the specification documents or the Design Bible, those documents win. Note the conflict in `docs/PROGRESS.md` under open questions.

2. Ground rules

These rules apply to every phase. They exist to keep CCOS sellable, auditable and consistent with the work already done.

Licensing and originality

- Do not copy code, schemas, prompts or UI text from the reference projects. Several are AGPL or GPL, which would force CCOS to be open-sourced. Implement from the requirements in this brief.
- Third-party libraries are allowed only under MIT, Apache-2.0, BSD or ISC licences. Record each new dependency and its licence in `docs/DEPENDENCIES.md`.

Data and money

- All quantities, prices and costs use `numeric` in Postgres and `decimal.js` in TypeScript. Never use JavaScript floats for money or quantities.
- Every tenant-owned table carries the organization and, where relevant, location scope used by the existing organization hierarchy, and has Row Level Security policies that call the existing RBAC+ABAC authorization functions.
- Add every new permission to the permissions seed, with sensible defaults for each existing role.
- Approvals use the existing approval policy and segregation-of-duties engine. Never build a second approval mechanism.

Records and history

- Stock and financial history is append-only. Corrections are reversing entries, never updates or deletes of posted rows. Enforce this with triggers, in the same style as the self-approval-ban trigger.
- Every create, update, status change and approval writes to the audit log (built in Phase 1).
- Business documents (requisitions, POs, receipts, invoices) get human-readable numbers from a per-organization sequence, e.g. `P0-2026-00042`.

Code and UI

- One forward-only migration per logical change, numbered after the existing five.
- Engine logic (costing, conversions, matching, valuation) lives in `packages/shared` as pure TypeScript with unit tests, so web, iOS and Edge Functions share it.
- New screens follow the Design Bible tokens and domain UI rules. All user-facing strings go through the i18n layer once it exists (Phase 5); until then, keep strings in one place per feature so they are easy to extract.
- AI features call the Claude API from Supabase Edge Functions only. Never expose keys to the client, and always show AI output for human review before it is posted.

3. Phase 1: Foundations

Phase 1 fixes cost accuracy and lays the stock, supplier and audit tables that every later phase depends on. Build it first even though it has the least visible UI.

3.1 Per-product unit conversions

Recipes use units like "bunch", "piece" or "egg" that cannot be costed without a product-specific weight or volume.

Table	Key columns
<code>product_unit_conversions</code>	<code>product_id</code> , <code>from_unit</code> , <code>to_unit</code> , <code>factor</code> (numeric), <code>note</code> , <code>unique</code> (<code>product_id</code> , <code>from_unit</code> , <code>to_unit</code>)

- The Cost Engine resolves units in this order: identical unit, global conversion (kg↔g, L↔ml), product-specific conversion, then a density-based conversion if the product has a density.
- If no path exists, the engine returns a typed `UnresolvedUnit` error. The recipe screen shows the line in a warning state instead of costing it at zero.
- Add a "missing conversions" report listing every recipe line that cannot be costed.

3.2 Product aliases and merge

Table	Key columns
<code>product_aliases</code>	<code>product_id</code> , <code>alias</code> (text), <code>source</code> (manual, import, supplier)

- Postgres function `merge_products(survivor_id, duplicate_id)` re-points every foreign key (recipe lines, supplier products, stock, conversions, aliases) to the survivor in one transaction, turns the duplicate's name into an alias, archives the duplicate and writes an audit entry.
- Merge requires a dedicated permission and shows a preview of affected recipes before confirming.
- Enable the `pg_trgm` extension with GIN indexes on product names and aliases. Search and import matching use similarity across both.

3.3 Supplier products and price history

One product can be bought from several suppliers in different pack sizes.

Table	Key columns
<code>suppliers</code>	name, tax_id, currency, payment_terms_days, contact fields, status (reuse if one already exists)
<code>supplier_products</code>	supplier_id, product_id, supplier_sku, pack_description (e.g. "case 12 × 1 L"), pack_qty_in_base_unit, purchase_unit, is_preferred, lead_time_days, min_order_qty
<code>supplier_prices</code>	supplier_product_id, price_per_pack, currency, valid_from, source (manual, invoice, quote) — append-only

- The current cost per base unit = latest valid price ÷ pack_qty_in_base_unit, from the preferred supplier unless the organization chooses "last purchase price" or "weighted average" as its costing method.
- A new price triggers recipe re-costing through the existing BFS dependency graph, and creates a notification when a product's cost moves more than a configurable threshold (default 5%).
- Product detail shows a price history chart per supplier.

3.4 Stock locations

Table	Key columns
<code>stock_locations</code>	parent_id, organization/location scope, name, type (outlet, store room, walk-in chiller, freezer, bar, dry store), is_active

Locations form a tree under each outlet in the existing organization hierarchy.

3.5 Stock movement ledger and batches

On-hand stock is never stored as an editable number. It is the sum of movements.

Table	Key columns
<code>stock_batches</code>	product_id, lot_code, received_at, expiry_date, supplier_id, unit_cost

Table	Key columns
<code>stock_movements</code>	<code>product_id</code> , <code>location_id</code> , <code>batch_id</code> (nullable), <code>qty</code> (signed, base unit), <code>unit_cost</code> , <code>movement_type</code> , <code>source_document_type</code> , <code>source_document_id</code> , <code>occurred_at</code> , <code>created_by</code> — append-only

- Movement types: receipt, transfer_out, transfer_in, waste, production_consume, production_output, sale_consume, stocktake_adjustment, return_to_supplier, opening_balance.
- A transfer is two movements sharing a `transfer_id`, posted in one transaction.
- Views: `stock_on_hand` (product × location × batch) and `stock_valuation` (on hand × cost under the organization's costing method).
- Rewire the existing low-stock detection to read from `stock_on_hand`.
- Waste entry screen: product, quantity, reason (spoilage, overproduction, dropped, expired, staff meal), optional photo.

3.6 Attachments

Table	Key columns
<code>attachments</code>	<code>entity_type</code> , <code>entity_id</code> , <code>storage_path</code> , <code>file_name</code> , <code>mime_type</code> , <code>uploaded_by</code>

Files live in a private Supabase Storage bucket with RLS that mirrors access to the parent entity. Build one reusable attachment component and add it to products, suppliers and recipes now; later phases reuse it.

3.7 Audit log

Table	Key columns
<code>audit_log</code>	<code>organization_id</code> , <code>entity_type</code> , <code>entity_id</code> , <code>action</code> , <code>actor_id</code> , <code>occurred_at</code> , <code>before</code> (jsonb), <code>after</code> (jsonb) — append-only

Write it from a generic trigger attached to every business table, consistent with the evidence/audit contract in the Platform Core specification. Add a history tab to product, recipe and supplier detail pages.

3.8 Custom fields

Table	Key columns
custom_field_definitions	organization_id, entity_type, key, label, field_type (text, number, date, select, boolean), options, required

Add a `custom_fields jsonb` column to products, suppliers and recipes, validated against the definitions. Forms render the extra fields automatically. This lets each customer extend CCOS without code changes.

4. Phase 2: Procure-to-pay

Phase 2 builds the full purchasing chain from requisition to payment-ready invoice. It reuses the existing approval policies, segregation-of-duties rules and self-approval ban, which is CCOS's main advantage over the reference projects.

4.1 Document chain

Each document has a header table, a lines table and a status. A shared `document_links` table records which document each line came from and how much quantity was carried forward, so partial receipts and partial invoicing work naturally.

Document	Header table	Lines table	Statuses
Purchase requisition (PR)	purchase_requisitions	purchase_requisition_lines	draft → submitted → approved / rejected → ordered / partially ordered → closed

Document	Header table	Lines table	Statuses
Purchase order (PO)	<code>purchase_orders</code>	<code>purchase_order_lines</code>	draft → submitted → approved / rejected → sent → partially received → received → closed / cancelled
Goods receipt note (GRN)	<code>goods_receipts</code>	<code>goods_receipt_lines</code>	draft → posted
Supplier invoice	<code>supplier_invoices</code>	<code>supplier_invoice_lines</code>	captured → matched / exception → approved for payment → paid
Supplier return	<code>supplier_returns</code>	<code>supplier_return_lines</code>	draft → posted → credited
Credit note	<code>supplier_credit_notes</code>	<code>supplier_credit_note_lines</code>	expected → received → applied

- Status changes go through one Postgres function per document type that checks the allowed transition and permissions, then writes the audit log. The client never sets `status` directly.
- PR and PO approvals use the existing approval policy engine, with thresholds by amount, cost centre and location. The self-approval ban applies.
- A PO can be created from one or more approved PRs, or directly by authorised roles. One PO = one supplier.
- POs can be exported as a branded PDF and emailed to the supplier. Sending an email always requires the user to press Send.

4.2 Goods receipt with quality checks

The GRN is where stock enters CCOS and where procurement meets food safety.

- Each line records ordered, received, rejected and short quantities, rejection reason, batch lot code, expiry date and unit cost.
- Chilled and frozen lines require a product temperature reading. Each line also has a packaging condition check and optional photos (Phase 1 attachments).
- Posting a GRN creates `stock_batches` and `receipt` movements in `stock_movements`, records the supplier price if it changed, and automatically drafts a supplier return for rejected quantities.
- The temperature reading is also written as a HACCP receiving log once Phase 3 exists. Design the GRN line so this link is a foreign key, not duplicated data.

4.3 Three-way match

A supplier invoice is checked line by line against the PO and the GRN before it can be approved for payment.

- Price check: invoice unit price vs PO unit price, within an organization-level tolerance (default $\pm 2\%$).
- Quantity check: invoiced quantity vs received quantity (not ordered quantity).
- Totals check: line totals, tax and invoice total recompute correctly.
- Duplicate check: same supplier and invoice number already exists.
- Matching logic lives in `packages/shared` as a pure function with unit tests covering exact match, price variance inside and outside tolerance, short delivery, over-invoicing, missing GRN and duplicates.
- Mismatched invoices go to an Exceptions queue showing each discrepancy side by side. A finance user resolves it by accepting the variance (with a reason), requesting a credit note, or rejecting the invoice. Every resolution is audited.

4.4 AI invoice capture

- Users upload a PDF or take a photo of a supplier invoice (web and iOS camera).
- An Edge Function sends it to the Claude API and asks for strict JSON: supplier name and tax ID, invoice number, dates, currency, lines (description, quantity, unit, unit price, tax), totals.
- Match the supplier by tax ID or fuzzy name, match lines to the supplier's products by SKU, then by `pg_trgm` similarity on description and aliases, and propose the open PO.
- Show a review screen with the original document beside the extracted fields. Low-confidence fields are highlighted. Nothing is saved as a supplier invoice until a user confirms.
- Organizations can define extra extraction fields through custom fields (Phase 1).

4.5 Auto-reorder

- Add `par_level` and `reorder_point` per product per location, in purchase units.
- A scheduled job (and a "Suggest order" button) finds products at or below the reorder point and drafts a PR per preferred supplier, topping up to par. It respects minimum order quantity and pack size.
- Drafts are never submitted automatically. A user reviews and submits them.

4.6 Request for quotation

- An RFQ lists products and quantities and is sent to several suppliers. Their quotes are entered manually or by AI capture.
- A comparison grid shows price per base unit, lead time and minimum order per supplier. The chosen quote converts into a PO in one click and records the prices in `supplier_prices` with source `quote`.

4.7 Tax

Tax rates are configurable per organization (e.g. Indonesian PPN, UAE VAT) and stored per line. Prices can be entered tax-inclusive or tax-exclusive.

5. Phase 3: HACCP module

Phase 3 builds the Quality and Compliance workspace. Open-source coverage here is thin, so this module is a differentiator for CCOS. Temperatures are stored in °C.

5.1 Monitoring points

Table	Key columns
<code>haccp_points</code>	location_id, name, category (receiving, cold storage, frozen storage, cooking, hot holding, cooling, reheating, cleaning, other), hazard, critical_min, critical_max, unit, frequency (per shift, daily, twice daily, weekly, per batch), responsible_role, is_ccp, is_active
<code>haccp_equipment</code>	location_id, name, type (fridge, freezer, hot cabinet, blast chiller, probe), serial, linked_point_id

Ship a starter template of common points (chillers $\leq 5^{\circ}\text{C}$, freezers $\leq -18^{\circ}\text{C}$, cooking core $\geq 75^{\circ}\text{C}$, hot holding $\geq 63^{\circ}\text{C}$) that an organization can adopt and edit. Make these limits editable because they differ by country.

5.2 Scheduled tasks

- A scheduler generates `haccp_tasks` from each point's frequency and the location's shift times.
- The kitchen tablet shows today's tasks by shift, with overdue ones flagged. Missed tasks remain visible and count against compliance.

5.3 Logs and corrective actions

Table	Key columns
<code>haccp_logs</code>	<code>point_id</code> , <code>task_id</code> , <code>value</code> , <code>recorded_at</code> , <code>recorded_by</code> , <code>source</code> (manual, sensor, receiving), <code>result</code> (pass, fail), <code>note</code> , <code>related_grn_line_id</code> — append-only
<code>haccp_corrective_actions</code>	<code>log_id</code> , <code>action_taken</code> (discard, re-chill, re-heat, adjust equipment, call technician, other), <code>description</code> , <code>product_disposition</code> , <code>recorded_by</code> , <code>verified_by</code> , <code>verified_at</code>

- Pass/fail is computed on save against the point's limits and shown immediately.
- A failed reading cannot be saved without a corrective action entry.
- A manager verifies each corrective action through the existing approval engine. The person who recorded it cannot verify it (self-approval ban).
- Alerts have a severity: critical (CCP failure), high (non-CCP failure), medium (task overdue), low (approaching a limit). Critical alerts notify the duty manager immediately.

5.4 Cooling logs

Cooling is a multi-stage process, so it needs its own log rather than a single reading.

- A cooling log records the product, batch, start time and start temperature, then readings at each stage.
- Default rule, editable per organization: from 60°C to 21°C within 2 hours, then to 5°C within the following 4 hours.
- The tablet shows countdown timers per stage and prompts for each reading. Breaching a stage creates a failed log and requires a corrective action.

5.5 Cleaning schedule

Table	Key columns
<code>cleaning_tasks</code>	<code>location_id</code> , <code>area</code> , <code>equipment_id</code> , <code>method</code> , <code>chemical</code> , <code>dilution</code> , <code>frequency</code> , <code>responsible_role</code>
<code>cleaning_records</code>	<code>cleaning_task_id</code> , <code>completed_at</code> , <code>completed_by</code> , <code>checked_by</code> , <code>note</code> , <code>photo attachment</code>

The same scheduler generates cleaning tasks. Show an overdue list per location.

5.6 Link to procurement

Every GRN temperature reading from Phase 2 automatically creates a `haccp_logs` row with source `receiving` and the GRN line linked. A failed receiving check blocks posting that line until it is rejected or a corrective action is recorded.

5.7 Reports and inspection pack

- Compliance score per location and period: $\text{tasks completed on time} \div \text{tasks due}$, plus count of failures and open corrective actions.
- One-click inspection pack as PDF and CSV for a date range: logs, failures, corrective actions with verifier, cleaning records and receiving checks.
- Traceability lookup: from a batch lot code, show the supplier, the GRN, its receiving check, and the recipes and production that used it.

5.8 Sensor readiness

Do not build sensor integrations yet. Do design for them: `source = sensor` is already allowed in `haccp_logs`, and an authenticated Edge Function endpoint accepts readings (`equipment serial`, `value`, `timestamp`) and writes them through the same pass/fail logic. Document the endpoint in `docs/`.

6. Phase 4: Stocktake, labels, scanning and recipe import

Phase 4 adds the daily operational tools that sit on top of Phases 1 to 3.

6.1 Stocktake sessions

Table	Key columns
stocktakes	location_id, type (full, partial, spot check), status (open → counting → submitted → approved → posted), snapshot_at
stocktake_lines	stocktake_id, product_id, batch_id, expected_qty (from snapshot), counted_qty, count_unit, counted_by

- Opening a stocktake freezes the expected quantities at `snapshot_at`. Movements after that time do not change the expected figures.
- Count sheets are ordered by location and shelf so staff walk the store in one pass. Counts can be entered in any purchase or recipe unit and are converted with Phase 1 conversions.
- The variance report shows expected vs counted quantity and value per line, sorted by absolute value impact.
- Approval runs through the approval engine. Posting creates `stocktake_adjustment` movements.
- Counting must work offline on the iOS app (see 7.6).

6.2 Barcode scanning

- Add `barcodes` to supplier products (one product can have several, e.g. unit and case).
- Use an Apache-2.0 or MIT barcode library (e.g. `zxing-js`) in the web app and Capacitor camera on iOS.
- Scanning works on the GRN, stocktake, waste and product search screens. An unknown barcode offers to link it to an existing supplier product.

6.3 Labels

- Label templates for prep labels (product, prepared on, use by, prepared by, allergens, batch), batch labels and shelf labels.
- Use-by dates come from a shelf-life field on products and recipes.
- Output as PDF for any printer, plus ZPL for Zebra label printers. Printing a prep label creates a `production_output` batch so it is traceable.

6.4 Recipe import

- Import from a URL by reading schema.org/Recipe JSON-LD, or from a photo or PDF through the Claude API.
- Parse each ingredient line into quantity, unit, ingredient and preparation note. Match the ingredient to products using `pg_trgm` over names and aliases.
- A review screen lists every line with its proposed product match and a confidence level. Unmatched lines can be linked, created as a new product, or saved as a new alias.
- Use the same import pipeline to load the original Excel workbooks as the first real dataset.

6.5 Search

A global search across products, recipes, suppliers and documents, using `pg_trgm` similarity so typos and partial names still match.

7. Phase 5: Refinements

Phase 5 adds the finance depth and platform features that make CCOS competitive for multi-outlet groups. Items can be built in any order.

7.1 Landed cost

- A landed cost entry links to one or more GRNs and lists extra charges: freight, import duty, customs, insurance, handling.
- Charges are allocated across GRN lines by value, quantity or weight (user's choice). Posting updates batch unit costs and therefore recipe costs.

7.2 Multi-currency

- Every document stores its currency and the exchange rate on the document date. Reports convert to the organization's base currency (IDR for the first customer).
- Rates are stored in an `exchange_rates` table (date, from, to, rate), entered manually or fetched daily by a scheduled job from a free rates source.

7.3 Budgets and committed spend

- `budgets` : cost centre, period, category, amount.
- Approved but not yet invoiced POs count as committed spend. The budget view shows budget, committed, actual and remaining.
- When a PO would exceed the remaining budget, the approval engine escalates it to the next approval level instead of blocking it outright. Make this configurable.

7.4 Accounts payable, payment runs and period lock

- AP aging report by supplier: current, 1–30, 31–60, 61–90 and 90+ days overdue, based on payment terms.
- Weekly payment proposal: invoices approved for payment and due, grouped by supplier, with credit notes netted off. Approving the proposal marks invoices as scheduled. Recording a payment marks them paid. CCOS never moves money itself.
- Closed periods: once a period is closed, a trigger blocks posting or editing documents dated in it. Reopening needs a dedicated permission and is audited.
- Do not build a general ledger. Generate journal entries per posted document and send them to Xero or QuickBooks through the integration hub, per the earlier decision. Until those adapters exist, provide a CSV journal export.

7.5 Supplier scorecard

Computed from Phase 2 documents per supplier and period: on-time delivery rate, fill rate (received ÷ ordered), price variance vs PO, rejection rate at receiving and invoice exception rate. Show it on the supplier page and use it in RFQ comparisons.

7.6 Offline-first on iOS

- Stocktake counting, waste entry, HACCP logs and cleaning records must work without a connection and sync when back online.
- Evaluate a Supabase-compatible sync layer with an Apache-2.0 or MIT licence (e.g. PowerSync) and record the choice as an ADR.
- Append-only tables make conflicts rare. For editable records, use last-write-wins and show the user any record that was overwritten.

7.7 Localisation

- Add an i18n layer (e.g. `react-i18next`) and move all strings into translation files.
- Ship English and Bahasa Indonesia first. Format numbers, currency and dates by locale.
- Each user picks their language. Kitchen-facing screens (tasks, logs, counts, labels) must be fully translated.

7.8 Webhooks

Emit events (PO approved, GRN posted, invoice approved, HACCP critical failure, stock below reorder point) to organization-configured webhook URLs, signed with a per-organization secret. This becomes the base of the integration hub.

8. Definition of done

A phase is done only when every item below is true. Claude Code ticks these off in `docs/PROGRESS.md` for each phase.

Every phase

- ☐ Migrations apply cleanly to a fresh database and on top of the existing schema.
- ☐ RLS policies exist on every new table and are tested with at least two organizations and two roles: a user must not see or change another organization's data.
- ☐ New permissions are added to the seed and assigned to roles.
- ☐ Unit tests pass for all new engine logic in `packages/shared`.
- ☐ Type-check and build pass for the web app and the iOS project.
- ☐ New screens follow the Design Bible and work at phone, tablet and desktop widths.
- ☐ The audit log records the new actions.
- ☐ Work is committed in logical commits with clear messages, `docs/PROGRESS.md` is updated, and any architecture decision has an ADR.

Phase checks

- ☐ Phase 1: a recipe using "1 bunch coriander" costs correctly via a product conversion; an unresolved unit shows a warning, not zero; merging two duplicate products keeps every recipe line intact; a new supplier price re-costs dependent recipes; on-hand stock equals the sum of movements.
- ☐ Phase 2: a PR → PO → partial GRN → second GRN → invoice flow works end to end; a price variance above tolerance lands in Exceptions; the PO creator cannot approve their own PO; rejected GRN quantities draft a supplier return; AI capture of a sample invoice produces a reviewable draft that is not saved until confirmed.
- ☐ Phase 3: a failed chiller reading cannot be saved without a corrective action; the recorder cannot verify their own corrective action; a cooling log breaching stage 1 creates a failure; a GRN temperature check appears in HACCP logs; the inspection pack exports for a date range; a lot code traces back to its supplier and GRN.
- ☐ Phase 4: a stocktake ignores movements after its snapshot; the variance report values match posted adjustments; scanning a barcode on the GRN selects the right supplier product; a printed prep label creates a traceable batch; a recipe imported from a URL reaches the review screen with product matches.
- ☐ Phase 5: landed cost raises batch and recipe costs; a document in a closed period cannot be edited; the AP aging buckets reconcile to approved unpaid invoices; offline counts sync without loss; the kitchen screens render fully in Bahasa Indonesia.

Reporting back

At the end of each phase, Claude Code writes a short summary in `docs/PROGRESS.md` : what was built, the migrations added, anything deferred and why, and open questions for Yves.